

Aula 30 - Blindagem de Dados e Representação de Objetos com PHP

Descrição: Exploração profunda do mecanismo de encapsulamento para garantir a integridade dos dados, implementação de Getters e Setters com validações e uso do método mágico `__toString()` para representação de objetos.

Antes de começarmos, pense: se você fosse o desenvolvedor responsável por um sistema de votação, qual seria o risco técnico de permitir que qualquer parte do código alterasse o contador de votos diretamente, sem passar por uma função de validação?

Encapsulamento e Níveis de Visibilidade

A. CONCEITO

Para entender o **Encapsulamento**, imagine que você está construindo um controle remoto. O usuário só precisa ver os botões (a interface); ele não deve mexer nos fios e circuitos internos, ou o controle vai parar de funcionar. No PHP, as "palavras-chave" de visibilidade são os cadeados que usamos para decidir quem pode mexer nos "fios" da nossa classe.

Os Três Níveis de Visibilidade

Imagine uma agência bancária para entender como o PHP separa os acessos:

- **Public (Público):** É o **Saguão do Banco**. Qualquer pessoa que entra na agência tem acesso. No código, qualquer arquivo do seu sistema pode ler ou alterar um atributo `public`.
- **Protected (Protegido):** É a **Área dos Funcionários**. Apenas quem trabalha lá (a própria classe) ou os estagiários e gerentes (classes filhas que herdaram os dados) podem entrar. O cliente (mundo externo) não entra aqui.
- **Private (Privado):** É o **Cofre Principal**. Só quem tem a chave mestra (a própria classe que criou o atributo) pode abrir. Nem mesmo as classes filhas (herança) conseguem mexer aqui. É o nível máximo de segurança.

Por que "blindar" os dados?

Se você deixa a senha de um usuário como `public`, qualquer parte do código pode fazer `$usuario->senha = "123"`. Isso quebra a **integridade**.

Com o encapsulamento, você tranca a senha como `private`. Se alguém quiser mudar a senha, terá que usar um método que você criou (como um `alterarSenha()`), onde você pode exigir a senha antiga e validar se a nova é forte o suficiente. **O objeto agora é o dono das próprias regras.**

A Grande Vantagem: Manutenção

Imagine que seu sistema tem 500 arquivos. Se o seu atributo é `public` e você resolve mudar o nome dele de `$valor` para `$preco`, você terá que procurar e alterar o nome nesses 500 arquivos.

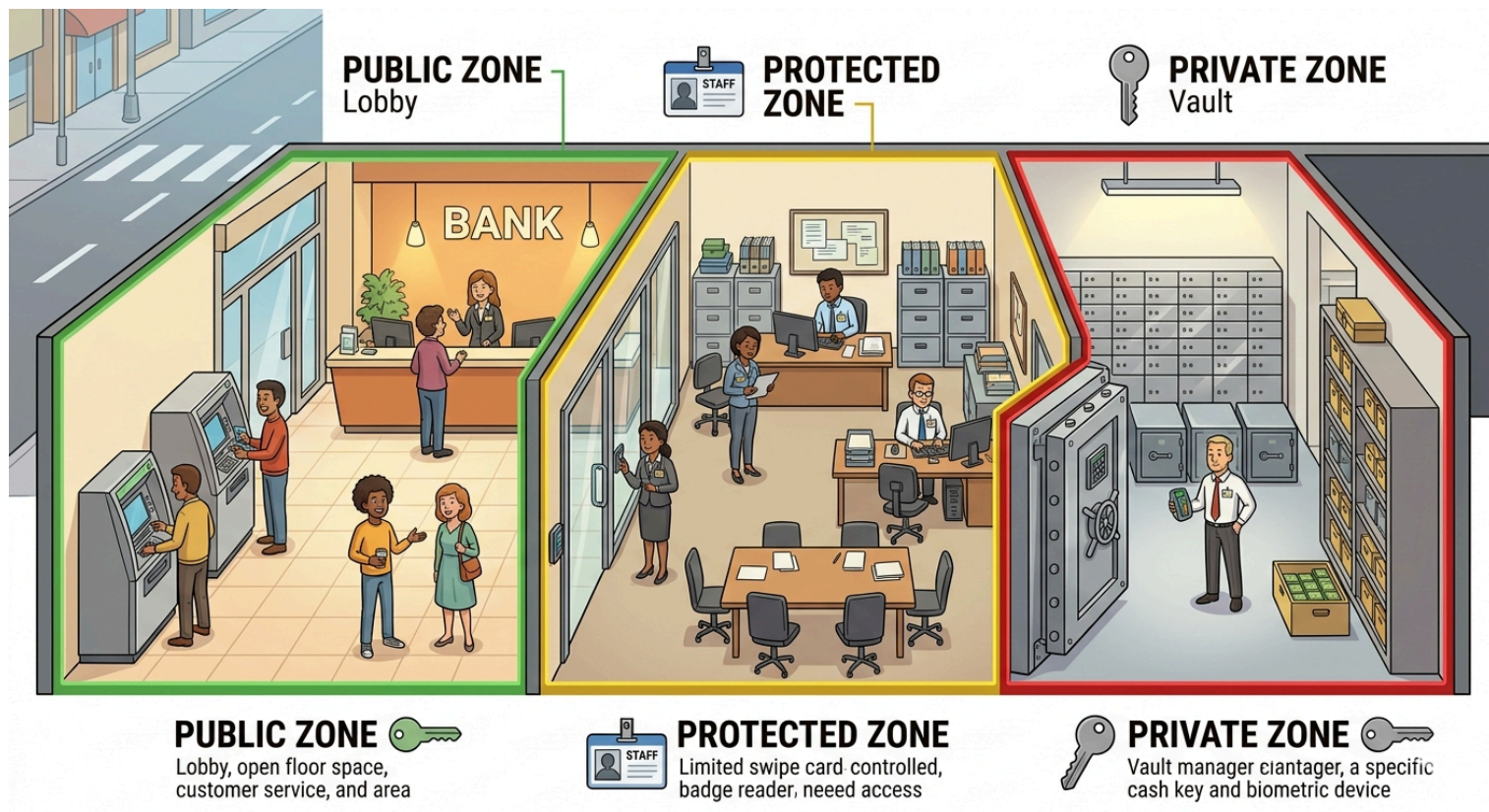
Se ele for `private`, você só precisa mudar **dentro daquela classe**.

Como o resto do sistema só conversa com ela através de "portas seguras" (métodos), ninguém lá fora precisa saber que você trocou o nome dos fios internos. Isso evita o famoso "efeito dominó", onde você conserta uma coisa e quebra dez.

B. VISUALIZANDO

Visibilidade	De fora da classe	Classes filhas (Herança)	Própria Classe
Public	✔ Sim	✔ Sim	✔ Sim

Protected	❌ Não	✅ Sim	✅ Sim
Private	❌ Não	❌ Não	✅ Sim



C. EXEMPLO DE CÓDIGO

PHP

```
<?php
```

```
class ContaBancaria {
```

```
// Atributos privados: ninguém altera o saldo diretamente de fora
```

```
private $saldo;
```

```
protected $titular;
```

```
public function __construct($titular, $valorInicial) {
```

```
    $this->titular = $titular;
```

```
// Atribuição interna permitida
```

```
    $this->saldo = $valorInicial;
```

```
}
```

```
// Método público: a "interface" segura para o mundo externo
```

```
public function depositar($valor) {
```

```
    if ($valor > 0) {
```

```
        $this->saldo += $valor;
```

```
        echo "Depósito de R$ $valor realizado com sucesso!<br>";
```

```
    }
```

```
}
```

```
}
```

```
$minhaConta = new ContaBancaria("João Silva", 100);
```

```
// $minhaConta->saldo = 5000; // ISSO GERARIA UM FATAL ERROR! O saldo está blindado.
```

```
$minhaConta->depositar(500); // Forma correta e segura
```

D. EXPLICAÇÃO DO CÓDIGO

- **private \$saldo**: O saldo está trancado. Tentar acessá-lo via `$obj->saldo` fora da classe interromperá a execução do programa com um erro.
- **protected \$titular**: O nome do titular pode ser usado em uma classe `ContaPoupanca` que herda desta, mas não pelo programador no arquivo `index.php`.
- **public function depositar**: Criamos uma regra. O saldo só aumenta se o valor for positivo. Essa é a essência do encapsulamento: **validação na entrada**.

E. ATENÇÃO! Um erro comum de iniciantes é colocar tudo como `public` para facilitar o desenvolvimento. Isso é uma dívida técnica perigosa. Lembre-se: **Seja restritivo por padrão**. Comece com `private` e só mude se houver uma necessidade real de herança (`protected`) ou acesso externo (`public`).

Métodos de Acesso (Getters e Setters)

A. CONCEITO

Para explicar esses conceitos de forma simples, vamos usar uma analogia do cotidiano: **o controle de uma encomenda em um prédio**. Imagine que os dados da sua classe (os atributos) são as encomendas que chegam. Se elas ficassem na calçada (**públicas**), qualquer um poderia mexer, levar ou estragar. Por isso, nós as guardamos dentro do prédio (**private**).

O que são Getters e Setters?

Eles funcionam como o **porteiro** do prédio. Você não entra no almoxarifado para pegar sua encomenda; você pede ao porteiro.

- **Setter (O Recebedor)**: É o método que "carrega" o valor para dentro do objeto. O nome sempre começa com `set` (do inglês, definir/colocar).
 - Exemplo: `setCpf($valor)`.
- **Getter (O Entregador)**: É o método que "pega" o valor de dentro e te mostra. O nome sempre começa com `get` (do inglês, pegar/obter).
 - Exemplo: `getCpf()`.

Por que usá-los? (A grande vantagem)

A maior vantagem é o **controle (validação)**. Se você deixa um atributo como público, alguém pode digitar um CPF com 20 números ou uma idade negativa. Com o **Setter**, você coloca uma "regra na porta":

Analogia do Porteiro: O Setter é como o porteiro conferindo se o entregador está com o documento certo antes de aceitar o pacote. Se o pacote estiver aberto ou sem etiqueta, ele rejeita.

Por que não usar atributos públicos?

Se você usar um atributo público (`public $cpf`), e depois de um ano decidir que o CPF deve ser salvo apenas com números (sem pontos e traços), você terá que abrir **todos** os arquivos do seu sistema e mudar onde o CPF é digitado.

Com o **Setter**, você muda apenas **uma linha** dentro da classe: você limpa os pontos dentro do método `setCPF()` e pronto! O resto do sistema continua funcionando sem nem saber que a regra mudou. Isso se chama **Encapsulamento**.

B. VISUALIZANDO

SETTER PORTER



O Porteiro (Setter) valida a identidade e verifica a permissão antes de abrir a porta.

GETTER PORTER



O Porteiro (Getter) informa quem está no prédio apenas para quem tem permissão.

C. EXEMPLO DE CÓDIGO

PHP

```
<?php
```

```
class Pessoa {
```

```
    private $nascimento;
```

```
    // Setter: Valida a data antes de gravar
```

```
    public function setNascimento($data) {
```

```
        $partes = explode('-', $data);
```

```
        // Verifica se a data é real (Ano, Mês, Dia)
```

```
        if (count($partes) == 3 && checkdate($partes[1], $partes[2], $partes[0])) {
```

```
            $this->nascimento = $data;
```

```
        } else {
```

```
            throw new Exception("Data de nascimento inválida!");
```

```
        }
```

```
    }
```

```
    // Getter: Apenas retorna o valor trancado
```

```
    public function getNascimento() {
```

```
        return $this->nascimento;
```

```
    }
```

```
}
```

```
?>
```

D. EXPLICAÇÃO DO CÓDIGO

- **setNascimento(\$data)**: Recebe uma string. Usa `explode` para separar os números e `checkdate` do PHP para garantir que não tentem salvar "31 de fevereiro".
- **throw new Exception**: Interrompe o processo se o dado for lixo. Isso evita que o banco de dados seja corrompido com informações erradas.

- **getNascimento()**: Fornece o acesso de leitura. Sem esse método, o valor ficaria inacessível para exibição.

E. DICA: No mercado, usamos Setters para aplicar "Sanitização". Se um usuário digita o nome com espaços extras ou tags HTML, o Setter pode limpar isso usando `trim()` ou `strip_tags()` antes de salvar no objeto.

Método Mágico `__toString()`

A. CONCEITO

Imagine que você tem uma caixa (o **Objeto**) cheia de ferramentas dentro. Se alguém te pedir "**me mostre essa caixa**" em formato de texto, você não pode simplesmente entregar a caixa inteira; você precisa descrever o que tem dentro ou mostrar uma etiqueta. O método `__toString()` é exatamente essa "**etiqueta de identificação**" do objeto.

O que é o `__toString()`?

No PHP, métodos que começam com dois underlines (`__`) são chamados de **Métodos Mágicos**. Eles são "mágicos" porque você não precisa chamá-los manualmente; o PHP os executa automaticamente em certas situações.

O `__toString()` é ativado toda vez que você tenta usar um objeto como se ele fosse uma string (texto).

- **Sem o método:** O PHP tenta imprimir o objeto, não consegue e trava o sistema com um erro fatal.
- **Com o método:** O PHP percebe que você quer um texto, entra na classe, executa o que está dentro do `__toString()` e exibe o resultado.

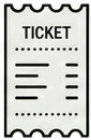
Por que ele é importante?

A. Evita erros catastróficos: Impede que seu site saia do ar só porque você esqueceu que `$usuario` era um objeto e tentou dar um `echo` nele.

B. Facilita a conversa entre sistemas (JSON): Hoje em dia, os sites enviam dados para aplicativos de celular ou para o JavaScript no navegador. O formato favorito para isso é o **JSON**. Se o seu objeto sabe se transformar em JSON sozinho através do `__toString()`, seu trabalho fica muito mais rápido.

Resumo da Regra: Sempre que você precisar que seu objeto "fale" (seja exibido como texto), use o `__toString()`. Ele é a voz do seu objeto!

B. VISUALIZANDO

AÇÃO: `echo`  `\$cliente;`		PRINT:  `\$titulo;`	
SEM __toString()	COM __toString()	SEM __toString()	COM __toString()
❌ PHP Fatal Error: Object of class Cliente could not be converted to string 		❌ Erro técnico: PHP Notice: Array to string conversion / Catchable Fatal Error 	
✅ "Cliente: Pablo Dall'Oglio" 		✅ {"id":1, "valor":250.00} 	

C. EXEMPLO DE CÓDIGO

```

PHP
<?php
class Titulo {
    private $dados = [];

    public function __construct($vencimento, $valor) {
        $this->dados['vencimento'] = $vencimento;
        $this->dados['valor'] = $valor;
    }

    // Método Mágico acionado automaticamente em contextos de string
    public function __toString() {
        // Retorna a representação textual do objeto em formato JSON
        return json_encode($this->dados);
    }
}

```

```

$contaLuz = new Titulo('2026-05-20', 150.50);
echo $contaLuz; // Saída: {"vencimento":"2026-05-20","valor":150.5}
?>

```

D. EXPLICAÇÃO DO CÓDIGO

- `private $dados = []`: Usamos um array interno para organizar as propriedades.
- `__toString()`: Note os dois sublinhados iniciais. O PHP entende que este é um método especial.
- `json_encode`: Transforma o array associativo em uma string padrão JSON.
- `echo $contaLuz`: O programador não precisa chamar `$contaLuz->exibir()`. O PHP "percebe" o eco e chama o `__toString` por baixo dos panos.

E. REFLITA: Se o seu objeto representa um Erro de Sistema, o que seria mais útil no `__toString()`: apenas a mensagem de erro ou a mensagem acompanhada da data e do arquivo onde o erro ocorreu?

MÃO NA MASSA

Objetivo: Construir uma classe de domínio `Produto` robusta, aplicando proteção de dados e integração.

Contexto: Vamos desenvolver o núcleo de um sistema de estoque. O produto não pode ter preço negativo, o estoque deve ser validado e, ao final, o sistema deve ser capaz de exportar os dados do produto para um sistema externo via JSON.

Tutorial Passo a Passo:

Passo 1: Estrutura Base e Blindagem

Crie o arquivo `Produto.class.php`. Declare os atributos como privados para evitar manipulação indevida.

```
PHP
class Produto {
    private $descricao;
    private $preco;
    private $estoque;
}
```

Passo 2: Implementando o Controle de Entrada (Setters)

Adicione validações para garantir que o estoque nunca seja negativo e o preço seja numérico.

```
PHP
public function setDescricao($valor) {
    $this->descricao = strtoupper($valor); // Força padrão em maiúsculas
}

public function setPreco($valor) {
    if (is_numeric($valor) && $valor > 0) {
        $this->preco = $valor;
    }
}
```

Passo 3: Acesso de Leitura (Getters)

Permita que o sistema leia os dados que foram validados.

```
PHP
public function getDescricao() { return $this->descricao; }
public function getPreco() { return $this->preco; }
```

Passo 4: Representação Mágica (Exportação)

Crie o `__toString` para facilitar a integração.

```
PHP
public function __toString() {
    return json_encode([
        "item" => $this->descricao,
        "valor" => $this->preco
    ]);
}
```

```
});  
}
```

Passo 5: Teste de Stress

Crie um arquivo `index.php` para testar se as travas funcionam.

PHP

```
$p = new Produto();  
$p->setDescricao("Teclado Mecânico");  
$p->setPreco(-50); // Não deve alterar nada  
echo "Produto: " . $p; // Deve mostrar o JSON
```

O Que Deve Ser Entregue?

Esta entrega simula um **ticket de desenvolvimento** em uma empresa real, onde não basta o código funcionar; ele precisa ser seguro e seguir os padrões da arquitetura orientada a objetos.

Você deve entregar um **pacote de software** contendo dois arquivos principais, organizados de forma que qualquer outro desenvolvedor da equipe consiga testar sua lógica.

1. O Modelo Técnico (`Produto.class.php`)

Este é o coração da sua entrega. O arquivo deve conter a classe `Produto` com os seguintes requisitos obrigatórios:

- **Atributos Privados:** As variáveis `$descricao`, `$preco` e `$estoque` **não podem** ser acessadas diretamente de fora da classe.
- **Setters Inteligentes:** Métodos que validam a entrada.
 - O preço não pode ser zero ou negativo.
 - A descrição deve ser salva automaticamente em letras maiúsculas (higienização de dados).
- **Getters:** Métodos públicos para leitura segura dos dados.
- **Representação Mágica:** O uso do método `__toString()` retornando uma string em formato **JSON**.

2. O Script de Teste (`index.php`)

Um arquivo simples para demonstrar que seu código é "à prova de erros". Ele deve:

- Instanciar um objeto da classe `Produto`.
- Tentar inserir um valor inválido (ex: preço negativo) e mostrar que o objeto **rejeitou** a alteração.
- Imprimir o objeto diretamente (`echo $p`) para mostrar o JSON funcionando.

Formato de Entrega

- **Arquivos:** `Produto.class.php` e `index.php`.
- **Onde:** Upload no ambiente virtual (AVA).
- **Dica Profissional:** Adicione um pequeno comentário no topo do arquivo com seu nome e a funcionalidade daquela classe. Isso faz parte da documentação técnica exigida no mercado.

FAQ - Perguntas Frequentes

Pergunta - Posso ter um Getter sem ter um Setter?

Resposta - Sim! Isso cria um atributo de "apenas leitura" (read-only). O valor pode ser definido apenas no construtor e depois apenas consultado.

Pergunta - Por que usar `protected` se o `private` é mais seguro?

Resposta - O `protected` é essencial para o reaproveitamento de código via Herança. Se você tem uma classe `Funcionario` e uma filha `Gerente`, o `Gerente` precisa acessar o atributo `$salario` para calcular bônus. Se for `private`, ele não consegue.

Pergunta - O PHP gera Getters e Setters automaticamente como o Java?

Resposta - Nativamente não, mas a maioria das IDEs (VS Code com extensões, PHPStorm) gera com um clique (Alt+Insert). Use a automação, mas entenda a lógica.

Pergunta - Posso usar IA (ChatGPT) para gerar meus Getters e Setters?

Resposta - Sim, é uma ótima ferramenta para evitar código repetitivo. No entanto, a IA costuma gerar Setters genéricos sem validação. **Sua função como dev é adicionar a regra de negócio** (ex: CPF válido, estoque positivo).

Pergunta - O `__toString` aceita argumentos?

Resposta - Não. Ele não pode receber nenhum parâmetro. Sua missão é apenas retornar a string do estado atual do objeto.

RESUMINDO

- **Encapsulamento:** Blindagem de atributos usando `private` ou `protected` para garantir a integridade dos dados. Aplique sempre.
- **Setters:** Portas de entrada. Servem para validar, limpar (sanitizar) e filtrar o que entra no objeto.
- **Getters:** Portas de saída. Servem para ler dados privados de forma controlada.
- **`__toString()`:** Método mágico que define a representação textual do objeto. Excelente para gerar JSON automaticamente.
- **Visibilidade Protected:** O "meio-termo" que permite que classes filhas acessem os dados, mas o mundo externo não.

FECHAMENTO

Conexão com a Situação de Aprendizagem (SA): Esta aula é o divisor de águas para o seu projeto final. No CRUD que estamos construindo, você não poderá mais passar dados "sujos" para o Banco de Dados. As classes que representam suas tabelas (Models) deverão usar **Encapsulamento** e **Setters** para validar e-mails, senhas e valores monetários antes mesmo de tentarem chegar ao banco, economizando processamento e evitando erros de integridade.